HTML Interview Preparation Guide (Fresher/Junior Handbook)

Use this as a **revise-fast handbook**. Each topic is written in an interview-friendly Q&A format with: **short answer + explanation + code + best practices**.

HTML Basics

Question: What is HTML and what is it used for?

Short Interview Answer: HTML (HyperText Markup Language) is the standard language used to structure content on the web (headings, paragraphs, links, forms, media).

Explanation: HTML describes the **structure** of a web page. Styling is handled by CSS and behavior by JavaScript. Browsers parse HTML into a DOM (Document Object Model).

Example Code:

```
<h1>Profile</h1>
<p>This is a short bio.</p>
<a href="/projects">View projects</a>
```

Best Practices:

- Use HTML for **structure and meaning**, not for styling.
- Prefer **semantic tags** for readability and accessibility.

Question: What is the difference between a Tag and an Element?

Short Interview Answer: A tag is the markup syntax (like `<p>`), while an element includes the opening tag, content, and closing tag (`<p>Hello</p>`).

Explanation: Tags are the angle-bracket parts. An element is the full node in the DOM (including attributes and content).

Example Code:

```
<p class="note">Hello World</p>
```

Best Practices:

- Speak precisely in interviews: "This **element** has a class attribute."

Question: What is the difference between Block and Inline elements?

Short Interview Answer: Block elements start on a new line and take full width (e.g., `div`, `p`). Inline elements flow within text and take only needed width (e.g., `span`, `a`).

Explanation: Block elements create layout "blocks." Inline elements are for small parts inside text. CSS can change display behavior.

Example Code:

```
<p>This is <span>inline text</span> inside a paragraph.</p>
<div>This is a block container</div>
```

Best Practices:

- Use block elements for layout structure and inline for text-level semantics.
- Don't wrap block elements inside inline elements (invalid in many cases).

Question: What are void (self-closing) elements?

Short Interview Answer: Void elements don't have closing tags or content (e.g., `img`, `input`, `br`, `meta`, `link`).

Explanation: They represent standalone nodes. In HTML5 you write `<img ...>` (not required to use `/>`).

Example Code:

```
<img src="avatar.png" alt="User avatar">
<input type="email" name="email">
<br>
```

Best Practices:

- Always provide `alt` for images (important for accessibility/SEO).
- Avoid using `<br>` for layout; use CSS.

HTML Document Structure

Question: What is `<!DOCTYPE html>` and why is it important?

Short Interview Answer: It tells the browser to use **standards mode** for HTML5 and avoids quirks mode rendering.

Explanation: Without it, browsers may behave inconsistently (quirks mode), causing CSS/layout issues.

Example Code:

```
<!DOCTYPE html>
<html lang="en">
  <head></head>
  <body></body>
</html>
```

Best Practices:

- Always include `<!DOCTYPE html>` at the top.

Question: What is the purpose of `<head>` vs `<body>`?

Short Interview Answer: `<head>` contains metadata (title, meta tags, CSS links, scripts), while `<body>` contains visible page content.

Explanation: Search engines and browsers use head data for preview/SEO/rendering. Body is the UI content.

Example Code:

```
<head>
  <meta charset="UTF-8">
  <title>Portfolio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Vishal Chandora</h1>
</body>
```

Best Practices:

- Keep SEO/meta tags in `<head>`.
- Load non-critical scripts with `defer`.

Question: Why is `<meta charset="UTF-8">` used?

Short Interview Answer: It sets the character encoding to UTF-8 so text and symbols render correctly.

Explanation: UTF-8 supports most characters worldwide. Placing it early helps the browser parse correctly.

Example Code:

```
<head>
  <meta charset="UTF-8">
</head>
```

Best Practices:

- Put charset meta tag as early as possible in `<head>`.

Question: What does `<meta name="viewport">` do?

Short Interview Answer: It controls layout on mobile devices; `width=device-width, initial-scale=1` makes pages responsive.

Explanation: Without it, mobile browsers may render pages at desktop width and scale down.

Example Code:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Best Practices:

- Always include viewport meta tag for responsive sites.

HTML Elements & Tags

Question: Difference between `<div>` and `<span>`?

Short Interview Answer: `div` is a block-level generic container; `span` is an inline generic container.

Explanation: Both have no semantic meaning by default. Use semantic tags when possible.

Example Code:

```
<div class="card">
  <span class="badge">New</span>
  <p>Item description</p>
</div>
```

Best Practices:

- Prefer semantic tags like `<section>`, `<article>`, `<nav>` over `<div>` when appropriate.

Question: What's the difference between `<strong>` and `<b>` (and `<em>` vs `<i>`)?

Short Interview Answer: `strong` and `em` convey semantic importance/emphasis; `b` and `i` are purely presentational.

Explanation: Screen readers may announce emphasis/importance for semantic tags.

Example Code:

```
<p><strong>Warning:</strong> This action is irreversible.</p>
<p>Please <em>read</em> the instructions carefully.</p>
```

Best Practices:

- Use `strong/em` when meaning matters; use CSS for styling.

Question: When would you use `<article>` vs `<section>`?

Short Interview Answer: `article` is self-contained content that can stand alone (blog post, news item). `section` groups related content within a page.

Explanation: Articles make sense if syndicated or reused independently. Sections organize a document.

Example Code:

```
<article>
  <h2>How I Built My Portfolio</h2>
  <p>...</p>
</article>

<section>
  <h2>Skills</h2>
  <p>...</p>
</section>
```

Best Practices:

- Use headings (`h2/h3...`) inside sections/articles for structure.

Attributes

Question: Difference between `id` and `class`?

Short Interview Answer: `id` must be unique on a page; `class` can be reused across many elements.

Explanation: IDs are often used for anchors and JS targeting; classes are used for styling and grouping.

Example Code:

```
<div id="profile">
  <p class="text-muted">Frontend Developer</p>
</div>
```

Best Practices:

- Don't reuse the same `id`.
- Prefer classes for CSS; avoid styling by id (harder to maintain).

Question: What are `data-*` attributes?

Short Interview Answer: Custom attributes to store extra information on elements, accessible via JS (`dataset`).

Explanation: They are valid HTML and useful for attaching metadata without abusing classes.

Example Code:

```
<button data-user-id="42" id="inviteBtn">Invite</button>

<script>
  const btn = document.getElementById("inviteBtn");
  console.log(btn.dataset.userId); // "42"
</script>
```

Best Practices:

- Store simple UI metadata; don't store secrets.
- Prefer structured state in JS for complex apps.

Question: What does `defer` do on scripts?

Short Interview Answer: `defer` loads the script in parallel and executes it after HTML parsing, preserving order.

Explanation: It prevents blocking page rendering and ensures DOM is ready.

Example Code:

```
<script src="app.js" defer></script>
```

Best Practices:

- Use `defer` for most scripts in `<head>`.
- Use `type="module"` (modules are deferred by default).

Semantic HTML

Question: What is Semantic HTML and why is it important?

Short Interview Answer: Semantic HTML uses meaningful tags (`header` , `nav` , `main` , `article`) to describe content, improving accessibility, SEO, and maintainability.

Explanation: Assistive tech and search engines understand page structure better. Teams can read code faster.

Example Code:

```
<header>
  <nav aria-label="Primary">
    <a href="/">Home</a>
  </nav>
</header>

<main>
  <article>
    <h1>Project</h1>
    <p>Details...</p>
  </article>
</main>

<footer>© 2026</footer>
```

Best Practices:

- Use one `<main>` per page.
- Use headings in logical order.

Question: What is the difference between `<header>` and `<head>`?

Short Interview Answer: `<head>` is metadata (not visible). `<header>` is a visible section header for a page or part of a page.

Explanation: `header` can appear multiple times (per section/article). `head` appears once per document.

Example Code:

```
<head><title>Site</title></head>
<body>
  <header><h1>Site Name</h1></header>
</body>
```

Best Practices:

- Don’t confuse these in interviews—very common trap.

Forms

Question: Why is the `label` tag important in forms?

Short Interview Answer: `label` improves accessibility and usability by connecting text to inputs; clicking the label focuses the input.

Explanation: Screen readers announce labels. Using `for` + `id` connects them.

Example Code:

```
<label for="email">Email</label>
<input id="email" name="email" type="email" required>
```

Best Practices:

- Every input should have a label (or `aria-label` as fallback).
- Use `required`, `type`, and `autocomplete` properly.

Question: Difference between GET and POST in forms?

Short Interview Answer: GET sends form data in the URL query string; POST sends it in the request body.

Explanation: GET is good for search/filter forms and is cacheable/bookmarkable. POST is better for sensitive or large data.

Example Code:

```
<form method="GET" action="/search">
  <input name="q">
</form>

<form method="POST" action="/register">
  <input name="email" type="email">
</form>
```

Best Practices:

- Don't use GET for passwords or sensitive info.
- Always validate on server even if HTML validation exists.

Question: What are common HTML form validations?

Short Interview Answer: `required`, input `type` (email/url), `min/max`, `minlength/maxlength`, `pattern`.

Explanation: Browser runs built-in validation before submit, but it's not a security feature.

Example Code:

```
<input type="password" required minlength="8">
<input type="text" pattern="[A-Za-z ]+">
```

Best Practices:

- Provide helpful error messages (with ARIA when needed).
- Validate again on backend.

Question: What is the difference between `button`, `submit`, and `reset` types?

Short Interview Answer: `type="submit"` submits the form, `type="reset"` resets fields, `type="button"` does nothing unless JS handles it.

Explanation: Default type for `<button>` inside forms is **submit** (common trap).

Example Code:

```
<button type="submit">Apply</button>
<button type="button">Preview</button>
<button type="reset">Clear</button>
```

Best Practices:

- Always set explicit `type` on `<button>` in forms.

Media (Images, Audio, Video)

Question: Why is the `alt` attribute important for images?

Short Interview Answer: `alt` provides text alternative for screen readers and displays when image fails; also helps SEO.

Explanation: For decorative images, use empty alt (`alt=""`) so screen readers skip it.

Example Code:

```
<img src="logo.png" alt="Company logo">
<img src="divider.png" alt="">
```

Best Practices:

- Meaningful images: descriptive alt.
- Decorative images: `alt=""` and preferably CSS background.

Question: How do you make images responsive?

Short Interview Answer: Use CSS like `max-width: 100%; height: auto;` and/or `srcset` for responsive images.

Explanation: `srcset` lets browser choose best resolution based on device.

Example Code:

```
<img
  src="photo-800.jpg"
  srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
  sizes="(max-width: 600px) 400px, 800px"
  alt="Team working together">
```

Best Practices:

- Use modern formats when possible (WebP/AVIF).
- Avoid huge images without `srcset`.

Question: How do `<audio>` and `<video>` work with `<source>`?

Short Interview Answer: You can provide multiple formats via `<source>` and browser picks the first supported.

Explanation: Useful for compatibility across browsers.

Example Code:

```
<video controls width="600">
  <source src="movie.webm" type="video/webm">
  <source src="movie.mp4" type="video/mp4">
  Sorry, your browser doesn’t support video.
</video>
```

Best Practices:

- Always provide controls (or accessible custom controls).
- Add captions/subtitles for accessibility when required.

Lists

Question: Difference between `<ul>`, `<ol>`, and `<dl>`?

Short Interview Answer: `ul` is unordered list, `ol` is ordered list, `dl` is description list (terms + descriptions).

Explanation: Use `ol` when order matters (steps). Use `dl` for key-value style info.

Example Code:

```
<ol>
  <li>Install Node</li>
  <li>Run the project</li>
</ol>

<dl>
  <dt>Role</dt>
  <dd>Frontend Developer</dd>
</dl>
```

Best Practices:

- Don't use lists just for indentation—use correct semantics.

Tables

Question: When should you use tables in HTML?

Short Interview Answer: Use tables only for **tabular data**, not layout.

Explanation: Tables create meaningful row/column relationships; using them for layout harms accessibility and responsiveness.

Example Code:

```
<table>
  <caption>Semester Results</caption>
  <thead>
    <tr><th>Subject</th><th>Grade</th></tr>
  </thead>
  <tbody>
    <tr><td>DBMS</td><td>A</td></tr>
  </tbody>
</table>
```

Best Practices:

- Use `<caption>`, `<thead>`, `<tbody>`, `<th>` for clarity.
- Use `scope="col"` / `scope="row"` for accessibility when helpful.

Question: Difference between `<th>` and `<td>`?

Short Interview Answer: `th` is a header cell (describes a row/column). `td` is a data cell.

Explanation: Screen readers use header cells to announce context for data cells.

Example Code:

```
<tr>
  <th scope="row">Total</th>
  <td>₹4,20,000</td>
</tr>
```

Best Practices:

- Use `scope` for better accessibility.

SEO & Meta Tags

Question: Which HTML tags are important for SEO?

Short Interview Answer: `<title>`, meta description, heading structure (`h1-h6`), semantic elements, `alt` text, canonical link.

Explanation: Search engines use title/description for snippets; headings and semantic content help relevance.

Example Code:

```
<head>
  <title>Vishal Chandora | Frontend Developer</title>
  <meta name="description" content="Frontend developer portfolio with Re
  <link rel="canonical" href="https://example.com/">
</head>
```

Best Practices:

- One clear `<h1>` per page (usually).
- Unique title + description per page.

Question: What is Open Graph and why use it?

Short Interview Answer: Open Graph meta tags control how links appear when shared on social platforms (title, image, description).

Explanation: Improves link previews and click-through rate.

Example Code:

```
<meta property="og:title" content="My Portfolio">
<meta property="og:description" content="Projects, skills, and contact."
<meta property="og:image" content="https://example.com/preview.png">
<meta property="og:url" content="https://example.com/">
```

Best Practices:

- Use absolute URLs for images.
- Provide Twitter card tags too when needed.

Accessibility

Question: What is ARIA and when should you use it?

Short Interview Answer: ARIA adds accessibility info when native HTML isn't enough, but should not replace semantic HTML.

Explanation: Use semantic elements first. ARIA is for custom widgets or when structure needs clarification.

Example Code:

```
<nav aria-label="Primary navigation">
  <a href="/jobs">Jobs</a>
</nav>
```

Best Practices:

- "No ARIA is better than bad ARIA."
- Prefer native controls (`button` , `input`) over div-based controls.

Question: What's the difference between `aria-label` and `aria-labelledby` ?

Short Interview Answer: `aria-label` provides direct text label; `aria-labelledby` references another element's text as the label.

Explanation: Use `aria-labelledby` when visible text exists; it avoids duplication and stays consistent.

Example Code:

```
<h2 id="settingsTitle">Settings</h2>
<section aria-labelledby="settingsTitle">
  ...
</section>
```

Best Practices:

- Don’t use both unless you know precedence and need it.

Question: Why is keyboard navigation important and how do you support it?

Short Interview Answer: Many users rely on keyboard; use proper focusable elements and logical tab order.

Explanation: Buttons/links are keyboard accessible by default. Divs are not.

Example Code:

```
<button type="button">Open modal</button>
<a href="/apply">Apply</a>
```

Best Practices:

- Don’t remove focus outlines without replacement.
- Avoid `tabindex` hacks; use native elements.

HTML5 Features

Question: What are some key HTML5 additions?

Short Interview Answer: Semantic tags (`main` , `section` , `article`), form input types, multimedia (`audio` , `video`), `canvas` , `data-*` .

Explanation: HTML5 improved semantics, native media, and forms—reducing reliance on plugins.

Example Code:

```
<input type="email" autocomplete="email" required>
<main>...</main>
<video controls src="demo.mp4"></video>
```

Best Practices:

- Use new input types for better mobile keyboards and validation.

Common Interview Traps (Very Important)

Question: What is the default `type` of `<button>` inside a form?

Short Interview Answer: Default is `submit`.

Explanation: Many bugs happen when a button unexpectedly submits a form.

Example Code:

```
<form>
  <button>Click</button> <!-- submits! -->
  <button type="button">Safe</button>
</form>
```

Best Practices:

- Always set `type="button"` for non-submit buttons.

Question: Is it valid to have multiple `<h1>` tags?

Short Interview Answer: Yes in HTML5 outline theory, but in practice many teams prefer one main `<h1>` per page for clarity/SEO.

Explanation: Browsers and SEO tools vary. A single clear page-level `<h1>` is safest.

Example Code:

```
<h1>Jobs</h1>
<section>
  <h2>Frontend</h2>
</section>
```

Best Practices:

- Keep heading hierarchy logical: don't jump `h2` to `h4`.

Question: Difference between `readonly` and `disabled`?

Short Interview Answer: `readonly` allows focus and form submission of value; `disabled` prevents focus and the value is not submitted.

Explanation: Disabled fields are ignored by form submit.

Example Code:

```
<input name="email" value="a@b.com" readonly>
<input name="token" value="123" disabled>
```

Best Practices:

- Use `readonly` when value should be submitted but not edited.

Question: Why is nesting a `<p>` inside another `<p>` wrong?

Short Interview Answer: `<p>` cannot contain another `<p>`; browser will auto-close tags causing unexpected DOM.

Explanation: HTML parser fixes invalid markup automatically, which may break styling/layout.

Example Code:

```
<!-- Wrong -->
<p>Outer <p>Inner</p></p>

<!-- Correct -->
<p>Outer</p>
<p>Inner</p>
```

Best Practices:

- Use validators when unsure.

Best Practices (Checklist)

Question: What are professional HTML best practices?

Short Interview Answer: Use semantic structure, accessible forms, correct headings, responsive meta, and valid markup.

Explanation: Good HTML improves accessibility, SEO, maintainability, and teamwork.

Example Code:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page</title>
</head>
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
</body>
</html>
```

Best Practices:

- Add `lang` on `<html>`.
- One `<main>` per page.
- Labels for inputs, alt for images.
- Avoid inline styles; keep HTML clean.

Rapid Revision Section (Quick Answers)

- **DOCTYPE:** Enables standards mode (HTML5).
- **Head vs Body:** Metadata vs visible content.
- **Tag vs Element:** Tag is syntax, element is full structure.
- **Block vs Inline:** New line/full width vs flows in text.
- **Void elements:** `img`, `input`, `br`, `meta`, `link`.
- **id vs class:** Unique vs reusable.
- **Semantic HTML:** Meaningful tags improve a11y/SEO.
- **Forms:** Always use `<label>`, set button types.
- **GET vs POST:** URL query vs request body.
- **alt:** Required for meaningful images; `alt=""` for decorative.
- **Viewport meta:** Required for responsive behavior.
- **Script defer:** Loads without blocking parse; executes after DOM parsed.
- **readonly vs disabled:** readonly submits; disabled doesn't.

Generated from markdown for sharing (PDF-friendly).